

Classification of images and enhancement of performance using parallel algorithm to detection of pneumonia

Gilberto de Melo, Sanderson O. Macedo, Silvio L. Vieira, and Leandro L. G. Oliveira

Abstract—Childhood pneumonia diagnosis using chest radiographies is a very important task performed by physicians with high impact on further clinical decisions and treatments. Pattern recognition techniques that would help automate the classification of chest radiographies into absence and presence of pneumonia with high reliability are being researched and tested, but it is still an open problem and this automate process have a high computational cost. This paper presents a CUDA-based parallel algorithm especially designed to provide speed gains for a wavelet feature extraction applied to high resolution DICOM images. The wavelet features are used for pneumonia detection and the proposed parallel technique improves the computing speed more than 12.75 times.

Keywords—Parallel algorithm, pneumonia detection, recognition techniques, CUDA.

I. INTRODUCTION

THE recent advent of faster and dedicated graphical processors developed parallel algorithms conception to new areas of applications. One of this is medical image analysis, whereupon signal and pattern recognition techniques are important research subjects, where parallel algorithms aim compression and feature analysis.

Since wavelet analysis has become an important tool parallel algorithms were developed for 1-D Fast Wavelet Transform [1], 2-D Wavelet Transform for images [2], [3], and for image compression [4].

This work presents a solution for problem of pneumonia detection in chest X-ray images [5] using wavelet coefficients. Presents a CUDA-based parallel algorithm that can speed up the feature vectors extraction for high resolution images. Also there are a classification of images using K-NN algorithm.

II. THEORETICAL BACKGROUND

A. Wavelet analysis

Wavelet analysis is a tool that provides a time and frequency information of the signal simultaneously. A wavelet decomposition of a signal helps to extract different informations of main signal, as high frequency parts or low frequency.

This study was financed in part by the Coordenacao de Aperfeicoamento de Pessoal de Nivel Superior - Brasil (CAPES) - Finance Code 001. Gilberto de Melo is with Federal University of Goias, Goiania, Brazil (e-mail: gilberto.melo@outlook.com). Sanderson O. Macedo is with Federal Institute of Goias, Goiania, Brazil (e-mail: sandecom@gmail.com). Silvio L. Vieira is with Federal University of Goias, Goiania, Brazil (e-mail: slvieira@ufg.br). Leandro L. G. Oliveira is with Federal University of Goias, Goiania, Brazil (e-mail: leandroluis@inf.ufg.br).

Digital images can be seen as bi-dimensional functions according to [6]. Equation (1) is a wavelet transform for a 2-D continuous function $f(x, y) \in L^2(R^2)$.

$$W_f(a, b_x, b_y) = \iint f(x, y) \psi_{a, b_x, b_y}(x, y) dx dy \quad (1)$$

2-D mother wavelet is shown in equation (2) [7].

$$\psi_{(a, b_x, b_y)}(x, y) = \frac{1}{|a|} \psi\left(\frac{x - b_x}{a}, \frac{y - b_y}{a}\right) \quad (2)$$

Wavelet Discrete Transform (WDT) is the transform for discrete functions. Haar Transform is known how the simplest basis functions of WDT. The 2-D analysis of this transform in discrete time filters the signal successively by bandpass filters, breaking it at each step in details [6].

Figure 1 presents decomposition of an original image (CA_i) (a). The decomposition produces a matrix (b) with four sub-matrices, being three with horizontal detail coefficients (CDh_{j+1}), verticals (CDv_{j+1}), diagonals (CDD_{j+1}) and one with the approximation coefficients (CA_{j+1}).

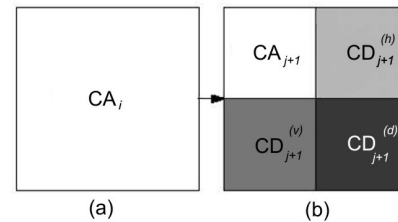


Fig. 1. Image Decomposition for a WDT.

B. CUDA-based parallel algorithm for Wavelet Discrete Transform

NVIDIA company introduced CUDA (Compute Unified Device Architecture) as a parallel computing platform and programming model implemented by GPUs (Graphical Processing Units) [8]. The CUDA framework has received great attention from the computing community because its potential for parallel processing. Each CUDA device is a set of GPU nuclei multiprocessors hosted in a CPU [9].

A parallel algorithm is isolated in a function called *kernel*. A *kernel* is organized by sets of *threads* blocks and separated *threads*. The *threads* are divided in blocks and they can be

synchronized and mutually cooperate by using a shared memory. The blocks of *threads* are identified by bi-dimensional coordinates *blockIdx.x* and *blockIdx.y*. The separated *threads* are identified by multidimensional coordinates *threadIdx.x*, *threadIdx.y* and *threadIdx.z*. In the CUDA framework *kernel* shall be processed using blocks and *threads*.

E.g., a NxN matrix (N=4) represented by a 16 positions vector, with respective indexes line by line as shown in Figure 2 (a). However, the data of each block for the columns processing in the vector will be in regions distant from each other as shown in Figure 2 (b). This form of access results in processing latency. For the columns it was proposed process the transpose of a matrix resulting from the lines processing (Figure 2 (c)). As final result of the wavelet transform to process the transpose of a matrix resulting from the columns processing (Figure 2 (d)).

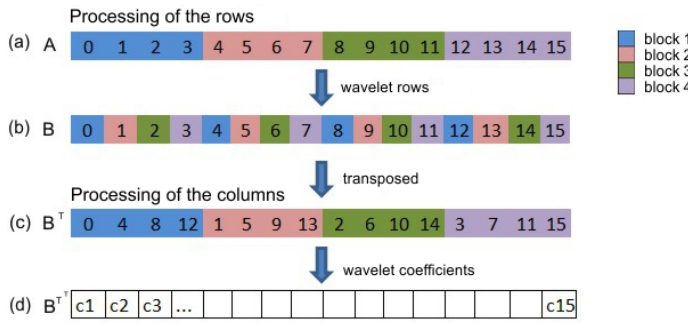


Fig. 2. Access processes of the shared memory showing the proposed scheme.

III. METHODOLOGY

In this work, it was proposed a experiment divided in two study of cases.

A. Study of Case 1 - Development of parallel algorithm to work with WDT

Images used in this application follows the DICOM (Digital Imaging and Communications in Medicine) standard and formats [10], which helps the exchange of data and information management for medical applications. Figure 3 shows two chest X-ray images of children under 5 years old with positive diagnosis of pneumonia. From them it can be seen the difficulty in providing the diagnosis, which is mainly based on a texture showing over one of the lung areas a different spatial and luminance configuration. Since scale, shape and intensity of these textures are not uniform and bones, other tissues and organs may appear overlaid and confuse the diagnosis.

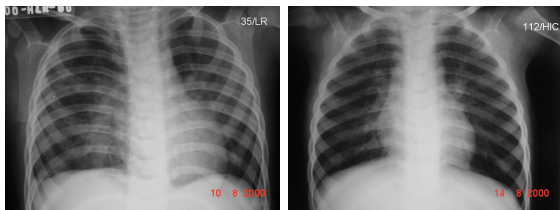


Fig. 3. Two images from our database with positive diagnosis of pneumonia in children under 5 years old.

It was designed two implementations for a Haar Wavelet Discrete Transform, serial and parallel, both applied to DICOM high resolution x-ray images. The proposition changes the way the wavelet transform algorithm access the shared memory of the CUDA in order to optimize the computing of the coefficients of the transform.

Figure 4 illustrate the serial version and the parallel algorithm proposed in CUDA with the optimization of access of the shared memory using matrices at the time of processing the columns. Matrices in CUDA are represented by concatenated vectors.

```

Serial algorithm
/*processing the rows*/
for(i=0; i<T;i++){
  for(j=0; j<T;j+=2,k++){
    int a = (origin[i][j]+origin[i][j+1])/2;
    int b = origin[i][j] - a;
    aux[i][k] = a;
    aux[i][k+w] = b;
  }
}
/*processing the columns*/
for(j=0; j<T;j++){
  for(i=0; i<T;i+=2,k++){
    int a = (aux[i][j]+aux[i+1][j])/2;
    int b = aux[i][j] - a;
    origin[k][j] = a;
    origin[k+w][j] = b;
  }
}

Parallel algorithm
/*threads access configuration*/
int w = dim*dim/2;
int idx = blockIdx.x*dim + threadIdx.x*2;
int idx2 = blockIdx.x + dim*threadIdx.x;

/*processing the rows*/
int aux = (int) (a[idx]+a[idx+1])/2;
int dif = a[idx]-aux;
b[idx2] = aux;
b[idx2+w] = dif;

/*processing the columns*/
aux = (int) (b[idx]+b[idx+1])/2;
dif = b[idx]-aux;
a[idx2] = aux;
a[idx2+w] = dif;

```

Fig. 4. Serial and parallel pieces of the wavelet discrete transform algorithm.

In the Haar Wavelet Discrete Transform it was proposed to process all the lines and later all the columns, where as the columns processing depend on the lines. If e.g., an image is NxN, in the CUDA-based proposed algorithm have N blocks, each processing a line of the matrix, and N/2 *threads* by block, with each block processing a column of the matrix. Each *thread* perform four operations, two (average and difference) using two values from lines and the other two (average and difference) using two values from the columns.

The main steps of the algorithm in CUDA were:

- 1) Copied the matrix with an image of memory host to the GPU memory;
- 2) Determined the configuration of the execution (N blocks by N/2 *threads*).
- 3) Ran the *kernel* to compute the elements of each line and column of the matrix;
- 4) Copied the result of the matrix of wavelet coefficients from GPU to CPU.

To do the comparison between the algorithm, it was selected three characteristics to know: difference variance, entropy and energy. Those characteristics were extracted by parallel algorithms using wavelets coefficients, they were generated from parallel wavelet transform applied x-ray images.

Equations (3, 4, 5) shows how were calculated the characteristics difference variance, entropy and energy.

Difference Variance:

$$difV = V(p_{x-y}) \quad (3)$$

Energy:

$$Ene = \sum_i \sum_j p[i, j]^2 \quad (4)$$

Entropy:

$$Ent = \sum_i \sum_j p[i, j] * \log(p[i, j]) \quad (5)$$

where:

- $p(i, j)$, (i,j)-nth probability of a gray level occurring in the image, $= \frac{P(i, j)}{N}$;
- $P(i, j)$, (i,j)-nth coefficient of wavelet decomposition;
- N , number of pixel in the image;
- $p_{x-y}(k)$, $\sum_i \sum_j p(i, j)$, where $k = |i - j|$.

Figure 5 shows parallelization of probability matrix, metric used to calculate the characteristics.

Serial algorithm <pre>for(int i=0; i<h;i++){ for(int j=0; j<w;j++){ p[i][j] = matrix[i][j]/N; } }</pre>	Parallel algorithm <pre>/*Thread access configuration*/ int idx = blockIdx.x*dim + threadIdx.x*2; p[idx] = matrix[idx]/dim*dim;</pre>
---	---

Fig. 5. Serial and Parallel algorithm versions of probability matrix.

Figures 6, 7, 8 shows parallelization of extraction algorithm of the characteristics difference variance, energy and entropy, respectively.

Serial algorithm <pre>// p is a matrix of probability MxN for(int i=0; i<M;i++){ for(int j=0; j<N;j++){ int k = Math.abs(i-j); diffPk[k] += p[i][j]; } } //average of p[i][j]; for(int i=0; i<diffPK.length;i++){ sum+=diffPK[i]; } avgPK = somatorio/diferenciaPK.length; //Variance of difference for(int i=0; j<diffPK.length;i++){ difVariance += (diffPK[i] - avgPK)/diffPK.length ; }</pre>	Parallel algorithm <pre>/*Thread access configuration*/ int idx = blockIdx.x*dim + threadIdx.x*2; /*Vector of difference*/ int k = blockIdx.x*dim - threadIdx.x*2; diffPk[k] +=p[idx]; //average avgPK = difPK[k]; //Variance of difference difVariance = (difPK[k] - avgPK)/blockIdx.x*dim;</pre>
---	--

Fig. 6. Serial and Parallel algorithm versions of difference variance.

Serial algorithm <pre>for(int i=0; i<h;i++){ for(int j=0; j<w;j++){ energy += p[i][j],2); } }</pre>	Parallel algorithm <pre>/*Thread access configuration*/ int idx = blockIdx.x*dim + threadIdx.x*2; energy += powf(p[idx],2);</pre>
---	---

Fig. 7. Serial and Parallel algorithm versions of energy.

Serial algorithm <pre>// p is a matrix of probability MxN for(int i=0; i<M;i++){ for(int j=0; j<N;j++){ entropy += p[i][j]*log(p[i,j]); } }</pre>	Parallel algorithm <pre>/*Thread access configuration*/ int idx = blockIdx.x*dim + threadIdx.x*2; entropy += p[idx]*log(p[idx]);</pre>
---	--

Fig. 8. Serial and Parallel algorithm versions entropy.

After the development of algorithms, the study tested the efficiency of parallel algorithms (wavelet transformation and yours characteristics) in relation of your serials versions.

Figure 9 shows the sequence of execution of parallel algorithm. The tests were with 5 DICOM resolutions: 512x512, 1024x1024, 2048x2048, 4096x4096, and 8192x8192 pixels.

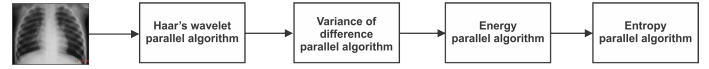


Fig. 9. Sequence of execution of parallel algorithms.

The serial part of the algorithm was run in a Dual Intel Pentium, each processor with clock 2.16 GHz and 4 GB RAM memory. This configuration was used as host for the GPU. The CUDA algorithm was run in a NVIDIA GeForce GTS 250, 1024 MB DDR3, GPU clock of 750 MHz, 128 Colors, 512 threads by block and coupled to host by PCI Express 2.0.

The efficiency analysis of a parallel algorithm can be verified by computing the speed up of the parallel version related to the serial version that show the equation (6).

$$S(p) = \frac{T(1)}{T(p)} \quad (6)$$

where: T(1) is time of execution with one processor and T(p) is the total time of data transfer from the CPU to the GPU and execution with p processors.

B. Study of Case 2 - Classification of Images using K-NN algorithm

In this study of case was selected 146 images of X-ray in high resolution of public system health of Goiania, Brazil. They were previously diagnosed by expertises in health, where 73 images with pneumonia and 73 images without. Each image were applied the parallel algorithms of wavelet transform and the characteristics extraction algorithms, resulting in a 9 positions vector.

Each characteristics vector was formed by combination of extraction characteristic of each sub-band wavelet decomposition. First 3 positions of storage vector were difference variance, in the 3 following were energy and finally 3 positions were entropy. Figure 10 shows this characteristics composition image.

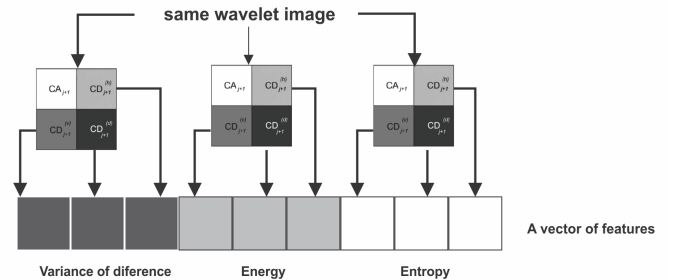


Fig. 10. Vector of features for each digital DICOM image.

The characteristics vector set were used by a classifier algorithm K-NN (k-Nearest Neighbor) to classify the images. This is a method of pattern recognition [11]. It is n-dimensional Euclidean space based. Each object is represented by a point in this space and the relation between them is given by Euclidean distance. A object is classified by majority neighbors vote, with objective to assign a new object inserted in Euclidean space. In this work the Euclidean space is defined by two

classes (without pneumonia = 0 and with pneumonia = 1) and each image were represented by characteristics vectors.

To validated the classification was used cross-validation. This method is a evaluation technique to evaluated the generalization capacity of models from a data set. It is largely applied in problems where the modeling goal is prediction [12]. In this project was used 10-fold cross-validation for the validation of the classification, where the sample set is divided into 10 subsets of approximately equal size. The 10–1 partitions are used in training a predictor, which is then tested on the remaining partition. This process is repeated 10 times, using in each cycle a different partition for testing. The final performance of the predictor is given by the average of the performances observed on each test subset [13].

IV. RESULTS AND DISCUSSION

Table I presents the results of study of case 1, the times, in seconds, and the speed up for serial and parallel executions. It was considered only one DICOM image in the different 5 resolutions defined for the CUDA-based parallel algorithms to speed the computation of the wavelet discrete transform and extraction of the features variance of the difference, energy and entropy. For the processing speed testing, it was not necessary to apply parallel algorithms in all 146 images, because the total processing time of 146 images is the processing time of one image times 146.

TABLE I
TIMES OF EXECUTION (SECONDS), AND SPEED UP GAINS, FOR SERIAL AND PARALLEL ALGORITHMS.

Resolution	Serial (1)	Total Serial (146)	CUDA(1)	Total CUDA (146)	Speed up
512x512	0.54	79.13	0.59	87.30	0.90
1024x1024	0.73	106.72	0.77	112.42	0.94
2048x2048	0.95	138.99	0.81	118.26	1.17
4096x4096	4.04	589.84	0.97	141.76	4.16
8192x8192	14.89	2173.94	1.16	170.38	12.75

It noticed that the parallel algorithm became more efficient when image resolution is higher (2048x2048, 4096x4096 and 8192x8192). Amdahl's law [14] states that the speed up of an algorithm using multiple processors in parallel computing is limited by the time needed by its serial portion to be run. However, since in this application with medical images higher resolution images are frequent and desired for detail analysis the speed up is much needed.

The speed up can be rewritten as:

$$S(p) \leq \frac{1}{f_s + \frac{1-f_s}{p}} \quad (7)$$

where: f_s is the serial portion of the parallel algorithm and p is the number of processors used.

It is possible see in results and as stated by Gustafson's law in [15], while the inputs grow in size the proportion of time spent by the serial part related to the parallel part diminish. And the speed up is incremented when the input size is larger than a specific value.

Figure 11 illustrates the times of execution by the image resolution for serial and parallel algorithms, and clearly the point where the speed up given by the proposed parallel

algorithm is advantageous. With 8192x8192 resolution the speed up is 7.4.

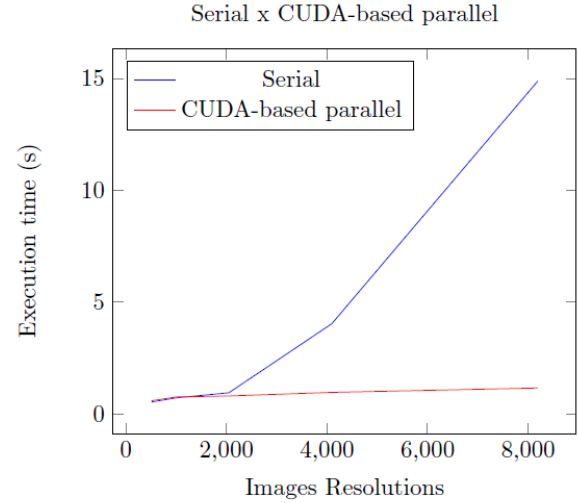


Fig. 11. Execution times for serial and CUDA-based parallel algorithms applied in a medical image.

As result of study of case 2, it was classified all 146 DICOM images using the technique cross-validation with k -fold = 10 and K-NN algorithm. For the classification, it was used a serial version of the K-NN algorithm.

Table II presents the accuracy of the classification obtained by the K-NN algorithm using the feature vectors of information extracted from the DICOM images by parallel algorithms.

TABLE II
RESULTS OF THE CLASSIFICATION BY CROSS-VALIDATION TECHNIQUE AND THE K-NN ALGORITHM.

Class	Correct Classification	Accuracy	Wrong Classification	Error
Pneumonia in	66 images	90.41 %	7 images	9.58 %
Pneumonia out	62 images	84.93 %	11 images	15.06 %
Average		87.67 %		12.29 %

To evaluated the level of accuracy and errors, it was considered informations given by pneumology studies area. They show the average false-positive results in city of Goiania are around of 30 % of according [16]. False-positive diagnoses contribute to the strengthening and proliferation of microorganisms that cause pneumonia, since the indiscriminate use of antibiotics leads to an increase in organisms that are resistant to the application of vaccines against the disease. In addition, such diagnoses raise public health costs by unnecessarily resources on medication and hospitalizations. In this study, it was obtained a mean of 87.67% correct classification and mean error scores of 12.29% as illustrates in Table II, a gain of 17.71% in relation to the misdiagnoses issued by health professionals in the city of Goiania.

V. CONCLUSION

In this work it was proposed two case studies. First a CUDA-based parallel algorithms to improve the time for processing a wavelet discrete transform and features extraction, which is the kernel for that application. The results presented

show that the CUDA-based algorithm proposal has reached speed up of 12.75 for 8192x8192 image resolution, and the gain is higher and worth starting from 2048x2048 resolution. Second study showed the classification of images using a classifier algorithm K-NN (k-Nearest Neighbor). The results presented 87.67% of correct classification, a gain of 17.71% in relation diagnoses of health professionals.

ACKNOWLEDGMENT

The authors would like to thank Coordination for the Improvement of Higher Education Personnel (Capes), the National Council for Scientific and Technological Development (CNPq) and Research Support Foundation of Goias State (FAPEG) for financial support research and scholarships.

REFERENCES

- [1] M. Holmström, "Parallelizing the fast wavelet transform," *Parallel Computing*, vol. 21, no. 11, pp. 1837–1848, 1995.
- [2] J. Franco, G. Bernabé, J. Fernández, and M. E. Acacio, "A parallel implementation of the 2d wavelet transform using cuda," in *Parallel, Distributed and Network-based Processing, 2009 17th Euromicro International Conference on*, pp. 111–118, IEEE, 2009.
- [3] M. Sultana and N. M. Mamun, "Simple and fast implementation of segmented matrix algorithm for haar dwt on a low cost gpu.," *International Journal on Signal & Image Processing*, vol. 3, no. 1, 2012.
- [4] K. Haapala, P. Kolinummi, T. Hamalainen, and J. Saarinen, "Parallel dsp implementation of wavelet transform in image compression," in *Circuits and Systems, 2000. Proceedings. ISCAS 2000 Geneva. The 2000 IEEE International Symposium on*, vol. 5, pp. 89–92, IEEE, 2000.
- [5] S. for double blind review, "Suppressed for double blind review," *International Journal of Medical Informatics*, 2008.
- [6] C. K. Chui, *Wavelets: a tutorial in theory and applications*, vol. 2. Academic Press, 2012.
- [7] I. Daubechies, *Ten lectures on wavelets*. Society for industrial and applied mathematics, 2006.
- [8] N. CUDA, "Cuda nvidia," jul 2014.
- [9] D. B. Kirk and W. H. Wen-mei, *Programming massively parallel processors: a hands-on approach*. Morgan Kaufmann, 2010.
- [10] DICOM, "Digital imaging and communication in medicine," abr 2014.
- [11] K. Feng, J. Gao, K. Feng, L. Liu, and Y. Li, "Active and passive nearest neighbor algorithm: A newly-developed supervised classifier," in *Advanced Intelligent Computing Theories and Applications. With Aspects of Artificial Intelligence*, pp. 189–196, Springer, 2012.
- [12] J. D. Rodriguez, A. Perez, and J. A. Lozano, "Sensitivity analysis of k-fold cross validation in prediction error estimation," *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, vol. 32, no. 3, pp. 569–575, 2010.
- [13] T. Fushiki, "Estimation of prediction error by using k-fold cross-validation," *Statistics and Computing*, vol. 21, no. 2, pp. 137–146, 2011.
- [14] X.-H. Sun and Y. Chen, "Reevaluating amdahl's law in the multicore era," *Journal of Parallel and Distributed Computing*, vol. 70, no. 2, pp. 183–188, 2010.
- [15] T. Huang, Y. Zhu, M. Qiu, X. Yin, and X. Wang, "Extending amdahl's law and gustafson's law by evaluating interconnections on multi-core processors," *The Journal of Supercomputing*, vol. 66, no. 1, pp. 305–319, 2013.
- [16] C. Franco, "Surveillance of pneumonias acquired in the community and admitted to pediatric hospitals of goiania," *Goiania: Federal University of Goias*, 2004.